

AI ENGINEER 2026 INTERVIEW PREP

205+ Tricky Interview Questions & Answers

Python • DSA • Docker • CI/CD • Kubernetes • MLflow/DVC • LLM • RAG • System Design

Compiled by Vaibhav | AI Engineer (Fresher)

TABLE OF CONTENTS

01	Python for AI Engineers	25 Qs
02	DSA (Data Structures & Algorithms)	25 Qs
03	Docker	20 Qs
04	CI/CD	20 Qs
05	Kubernetes	20 Qs
06	MLflow & DVC (Experiment Tracking / Versioning)	20 Qs
07	LLM Fundamentals	25 Qs
08	RAG (Retrieval-Augmented Generation)	25 Qs
09	System Design for AI/LLM Applications	25 Qs

Total: 205 Questions across 9 categories

Q1. What is the GIL and why does it matter for AI workloads?

Ans: Global Interpreter Lock allows only one thread to execute Python bytecode at a time. CPU-bound ML code doesn't benefit from threading; use multiprocessing or C-extensions (numpy) that release the GIL.

Q2. Difference between deepcopy and shallow copy in Python?

Ans: Shallow copy copies references to nested objects (changes propagate); deepcopy recursively copies everything, creating fully independent objects.

Q3. How do generators help when building data pipelines for LLM training?

Ans: Generators yield one item at a time, avoiding loading full datasets into memory - critical for streaming huge token/text corpora.

Q4. Explain *args and **kwargs with a real use case in an ML API.

Ans: *args collects positional args as a tuple, **kwargs collects keyword args as a dict - used in FastAPI wrapper functions to forward arbitrary model parameters.

Q5. What is the difference between @staticmethod, @classmethod and instance methods?

Ans: Instance methods take self and access instance state; classmethod takes cls and works on class-level state; staticmethod takes neither, it's a plain function namespaced under the class.

Q6. How does Python's async/await help in serving LLM APIs with FastAPI?

Ans: async/await lets I/O-bound calls (DB, external LLM API) yield control instead of blocking, so a single worker can handle many concurrent requests waiting on network calls.

Q7. What's a decorator and how would you build one for logging LLM latency?

Ans: A decorator wraps a function to add behavior. You'd wrap the LLM call function, capture time before/after execution, and log the delta without touching the original function's code.

Q8. Explain Python memory management and reference counting.

Ans: CPython uses reference counting plus a cyclic garbage collector; objects are freed when refcount hits zero, and gc handles reference cycles refcounting alone can't clear.

Q9. Difference between list, tuple, set and dict in terms of use in feature engineering?

Ans: List is ordered/mutable (sequences), tuple is ordered/immutable (fixed records), set is unordered/unique (deduplication), dict is key-value (fast lookups like feature maps).

Q10. What is monkey patching and would you ever use it in production ML code?

Ans: Monkey patching modifies a class/module at runtime. Useful for testing/mocking but risky in production since it breaks predictability and traceability.

Q11. Explain Python's context managers (with statement) and their use with model checkpoints.

Ans: Context managers guarantee setup/teardown via `__enter__`/`__exit__` - used to safely open, load and close large model checkpoint files even if an exception occurs.

Q12. How do you handle circular imports in a large ML codebase?

Ans: Restructure modules, use local imports inside functions, or move shared code to a separate common module to break the cycle.

Q13. What is the difference between `__init__` and `__new__`?

Ans: `__new__` creates and returns a new instance (allocation); `__init__` initializes the already-created instance (setting attributes).

Q14. Explain Python's duck typing and how it helps in writing flexible agent tools.

Ans: If an object implements the expected methods/attributes, its actual type doesn't matter - lets you swap LLM providers or tool implementations without rigid interfaces.

Q15. How would you profile a slow data preprocessing pipeline?

Ans: Use `cProfile`/`line_profiler` to find hotspots, `memory_profiler` for memory, and vectorize with `numpy`/`pandas` instead of Python loops where bottlenecks appear.

Q16. Difference between multiprocessing and multithreading for a RAG ingestion pipeline?

Ans: Multithreading suits I/O-bound tasks (API calls, file reads); multiprocessing suits CPU-bound tasks (embedding generation, chunking) since it bypasses the GIL using separate processes.

Q17. What are Python type hints and why do they matter in production agent code?

Ans: Type hints (typing module) document expected types, enable static checking (mypy) and better IDE support - reduces runtime bugs in complex agent pipelines.

Q18. Explain the difference between `==` and `is` in Python.

Ans: `==` compares value equality; `is` compares object identity (same memory reference).

Q19. How does Python handle exceptions in async code differently from sync code?

Ans: In async, exceptions raised in a coroutine propagate to the awaiting caller; unawaited tasks can swallow exceptions silently unless you check `task.exception()`.

Q20. What is a Python metaclass and have you ever used one in an ML framework?

Ans: A metaclass controls class creation itself (class of a class). Frameworks like Pydantic use metaclasses to auto-generate validation logic from class attributes.

Q21. Explain lazy evaluation and where it's used in pandas/numpy pipelines.

Ans: Lazy evaluation delays computation until the result is needed - similar concept in Dask/Polars query planning, letting the engine optimize the whole pipeline before executing.

Q22. What's the difference between a Python module and a package?

Ans: A module is a single .py file; a package is a directory of modules containing an `__init__.py` (or namespace package) that groups related modules.

Q23. How would you implement a simple LRU cache for repeated LLM prompt calls?

Ans: Use `functools.lru_cache` decorator or an `OrderedDict` that evicts the least-recently-used entry once capacity is exceeded - saves redundant API calls for repeated prompts.

Q24. Explain Python's Global vs Local vs Enclosing vs Built-in (LEGB) scope resolution.

Ans: Python resolves names by checking Local scope first, then Enclosing (closures), then Global (module level), then Built-in - in that order.

Q25. Why would you use dataclasses over plain classes for structuring LLM response schemas?

Ans: Dataclasses auto-generate `__init__`, `__repr__`, `__eq__` reducing boilerplate, and pair well with type hints for structured, readable data containers like API response models.

Q1. How would you detect a cycle in a directed graph (relevant to agent workflow graphs)?

Ans: Use DFS with three states (white/gray/black) or track a recursion stack; if you revisit a node currently in the stack, a cycle exists - this is exactly how LangGraph validates workflow graphs.

Q2. Explain time complexity of vector similarity search using brute force vs ANN.

Ans: Brute force is $O(n*d)$ per query (n vectors, d dimensions); Approximate Nearest Neighbor (HNSW, IVF) reduces this to roughly $O(\log n)$ at the cost of exact accuracy.

Q3. How would you implement a sliding window for token-based text chunking?

Ans: Maintain start/end pointers over the token array, move forward by (`chunk_size - overlap`) each iteration, slicing tokens between pointers into a chunk.

Q4. What data structure would you use to implement an LRU cache and why?

Ans: A hash map plus a doubly linked list - hash map gives $O(1)$ lookup, doubly linked list gives $O(1)$ reordering/eviction of least recently used entries.

Q5. Explain Big-O of building an inverted index for a BM25 search.

Ans: Building it is roughly $O(N*L)$ where N is number of documents and L average document length, since every token in every doc is inserted once into the index.

Q6. How do you find the top-k most similar embeddings efficiently?

Ans: Use a min-heap of size k while scanning scores - $O(n \log k)$ instead of sorting all n scores which is $O(n \log n)$.

Q7. Explain how a trie could optimize autocomplete for a prompt-suggestion feature.

Ans: A trie stores common prefixes once, so lookups/prefix matches are $O(m)$ where m is the query length, independent of total dictionary size.

Q8. What's the difference between BFS and DFS, and when would each suit a multi-agent tool graph traversal?

Ans: BFS explores level by level (shortest path, good for finding nearest matching agent), DFS explores depth-first (good for exhaustive dependency resolution in a workflow).

Q9. How would you design a rate limiter using a sliding window algorithm?

Ans: Store timestamps of requests in a queue/counter buckets; on each new request, drop timestamps older than the window and check if count is within the allowed limit.

Q10. Explain the time complexity difference between hash map and binary search tree lookups.

Ans: Hash map average $O(1)$ lookup but $O(n)$ worst case (collisions); balanced BST guarantees $O(\log n)$ lookup and preserves sorted order which hash maps don't.

Q11. How would you merge overlapping time intervals (e.g. GPU job scheduling)?

Ans: Sort intervals by start time, then iterate and merge the current interval into the last one in the result if their ranges overlap, else push a new interval.

Q12. Explain how a min-heap based priority queue would help in Dijkstra's algorithm for optimal agent routing.

Ans: The heap always pops the currently cheapest reachable node in $O(\log n)$, letting Dijkstra greedily expand the shortest known path first.

Q13. What is dynamic programming, and how would you apply it to optimal text summarization segment selection?

Ans: DP breaks a problem into overlapping subproblems and caches results - here, choosing the best combination of segments under a token budget resembles a knapsack DP.

Q14. How would you check if a binary tree is balanced (relevant to indexing structures)?

Ans: Recursively compute height of each subtree, returning early with a failure flag if any subtree's height difference exceeds 1 - $O(n)$ overall.

Q15. Explain topological sort and its use in DAG-based agent pipelines (like LangGraph/Airflow).

Ans: Topological sort orders nodes so every directed edge goes from earlier to later in the sequence - it determines valid execution order of dependent pipeline/agent steps.

Q16. How would you implement a two-pointer approach for deduplicating retrieved chunks?

Ans: Sort by a similarity/ID key, then use two pointers to compare adjacent elements, skipping/merging near-duplicates in a single linear pass.

Q17. What's the difference between an array and a linked list for implementing a conversation history buffer?

Ans: Array gives $O(1)$ random access but $O(n)$ insert/delete; linked list gives $O(1)$ insert/delete at the ends but $O(n)$ random access - a deque fits conversation buffers well.

Q18. Explain quantization from an algorithmic complexity view (not ML) - how does binary search relate to reducing float precision buckets?

Ans: Binary search over threshold buckets can efficiently map a continuous float to its nearest quantization bin in $O(\log k)$ for k bins instead of linear scanning.

Q19. How would you find the median of a streaming set of latency values (for monitoring)?

Ans: Maintain two heaps - a max-heap for the lower half and a min-heap for the upper half - keeping them balanced so the median is always at the top in $O(\log n)$ per insert.

Q20. Explain the knapsack problem and its relevance to selecting context chunks under a token limit.

Ans: Knapsack picks the subset of items maximizing value under a weight constraint - directly analogous to selecting the most relevant chunks that maximize retrieved information under a fixed token budget.

Q21. How would you detect duplicate documents before embedding (near-duplicate detection)?

Ans: Use MinHash + LSH (Locality Sensitive Hashing) to estimate Jaccard similarity efficiently without pairwise $O(n^2)$ comparison.

Q22. What is the time complexity of quicksort vs mergesort, and which would you pick for sorting large embedding score arrays?

Ans: Quicksort averages $O(n \log n)$ but worst case $O(n^2)$; mergesort guarantees $O(n \log n)$ with extra $O(n)$ space - mergesort is safer for large, unpredictable score distributions.

Q23. Explain a hash set based approach to deduplicate tool call arguments in an agent loop.

Ans: Hash the serialized arguments (e.g. JSON string) and store in a set; skip re-executing a tool call if its argument hash was already seen.

Q24. How would you implement backtracking for exploring multiple agent tool-call paths?

Ans: Recursively try a tool call path, and if it fails to satisfy the goal, undo/rollback state and try the next candidate path - classic backtracking with pruning.

Q25. Explain amortized time complexity using Python's list append as an example.

Ans: Appending is normally $O(1)$, but occasionally the underlying array must resize and copy - $O(n)$ - however this cost averaged (amortized) over many appends is still $O(1)$.

Q1. What's the difference between a Docker image and a container?

Ans: An image is a read-only blueprint/template; a container is a running (or stopped) instance of that image with its own writable layer.

Q2. Why would you use multi-stage builds for an LLM app's Dockerfile?

Ans: Multi-stage builds let you compile/install heavy build dependencies in one stage and copy only the final artifacts into a slim runtime stage, drastically reducing final image size.

Q3. How do you reduce a Python ML Docker image size (e.g. from 3GB to under 1GB)?

Ans: Use slim/alpine base images, multi-stage builds, `--no-cache-dir` with pip, remove build tools after compiling, and only copy needed files instead of the whole context.

Q4. Explain the difference between CMD and ENTRYPOINT in a Dockerfile.

Ans: ENTRYPOINT defines the fixed executable that always runs; CMD supplies default arguments that can be overridden at runtime - together they define the container's default behavior.

Q5. What is a Docker layer and why does layer caching matter for CI build speed?

Ans: Each Dockerfile instruction creates a cached layer; unchanged layers are reused on rebuild, so ordering (e.g. installing dependencies before copying code) speeds up rebuilds significantly.

Q6. How would you persist vector database data (like Chroma/Qdrant) across container restarts?

Ans: Mount a named volume or bind mount to the container's data directory so the DB files survive even if the container is removed and recreated.

Q7. What's the difference between COPY and ADD in a Dockerfile?

Ans: COPY simply copies files/directories; ADD additionally supports remote URLs and auto-extracts local tar archives - COPY is preferred for predictability.

Q8. How do you pass secrets (like GROQ_API_KEY) into a container securely?

Ans: Use environment variables injected at runtime (`docker run -e / env_file`), Docker secrets, or a secrets manager - never hardcode keys inside the image layers.

Q9. Explain Docker networking modes (bridge, host, none) in context of a multi-container agent app.

Ans: Bridge (default) isolates containers on a private network with port mapping; host shares the host's network stack directly; none disables networking entirely - agent microservices typically use bridge with a custom network for service discovery.

Q10. What is Docker Compose and why use it over multiple docker run commands for a RAG stack?

Ans: Compose defines all services (API, vector DB, Redis, frontend) with their networks/volumes in one YAML file, letting you spin the entire stack up/down with a single command.

Q11. How would you debug a container that keeps restarting (CrashLoop)?

Ans: Check docker logs , run the image interactively with `--entrypoint /bin/sh`, inspect exit codes, and verify environment variables/health checks are correctly configured.

Q12. Explain the difference between a bind mount and a named volume.

Ans: Bind mounts link a specific host path directly into the container (host-managed); named volumes are managed by Docker itself, more portable and preferred for production.

Q13. How do you optimize Docker build cache when requirements.txt changes frequently?

Ans: Copy requirements.txt and run pip install before copying the rest of the source code, so dependency layers stay cached unless requirements actually change.

Q14. What is a healthcheck in Docker and how would you configure one for a FastAPI LLM service?

Ans: HEALTHCHECK runs a periodic command (e.g. curl the /health endpoint) inside the container; Docker marks it unhealthy if it fails repeatedly, which orchestrators use for auto-restart decisions.

Q15. Explain the difference between docker exec and docker attach.

Ans: docker exec runs a new process inside a running container (e.g. opening a new shell); docker attach connects to the container's existing primary process's stdin/stdout/stderr.

Q16. How would you containerize a GPU-based embedding model?

Ans: Use an NVIDIA CUDA base image, install the nvidia-container-toolkit on the host, and run with `--gpus all` so the container can access the host's GPU.

Q17. What's the purpose of a .dockerignore file in an AI project?

Ans: It excludes unnecessary files (venvs, __pycache__, large datasets, .git) from the build context, speeding up builds and shrinking image size.

Q18. Explain how you'd scale a Dockerized FastAPI LLM backend horizontally without Kubernetes.

Ans: Run multiple container replicas behind a reverse proxy/load balancer (e.g. Nginx or Docker Swarm mode) that distributes incoming requests across them.

Q19. What is the difference between ENV and ARG in a Dockerfile?

Ans: ARG is only available during the build process (build-time variable); ENV persists into the running container and is accessible at runtime.

Q20. How would you troubleshoot 'no space left on device' errors during Docker builds?

Ans: Run docker system prune to remove unused images/containers/build cache, and check for excessive dangling layers from repeated failed builds.

Q1. What's the difference between Continuous Integration and Continuous Deployment?

Ans: CI focuses on automatically building and testing code on every commit to catch issues early; CD extends this by automatically releasing that validated build to production (or staging) environments.

Q2. How would you design a CI/CD pipeline for an LLM-powered FastAPI microservice?

Ans: Stages: lint/format check, unit tests, build Docker image, run integration tests against a staging vector DB, push image to registry, then deploy via rolling update to the target environment.

Q3. What are the risks of auto-deploying an LLM app to production without a staging gate?

Ans: Prompt regressions, cost spikes from unexpected token usage, and breaking changes to output schema could reach users directly - a staging/canary gate catches these before full rollout.

Q4. Explain blue-green deployment vs canary deployment for an AI service.

Ans: Blue-green switches all traffic instantly between two full environments (fast rollback); canary gradually shifts a small percentage of traffic to the new version, reducing blast radius if something's wrong.

Q5. How do you run automated evaluation (accuracy/hallucination checks) as part of a CI pipeline for RAG?

Ans: Add an eval stage that runs a fixed test set of Q&A; pairs through the pipeline, scores outputs (using an LLM-as-judge or exact-match metrics), and fails the build if scores drop below a threshold.

Q6. What is a CI/CD pipeline gate and how would you use one to prevent bad prompts from shipping?

Ans: A gate is an automated check that must pass before proceeding; you'd add a prompt-regression test suite as a required gate before merging changes to prompt templates.

Q7. Explain GitHub Actions workflow triggers relevant to an ML repo (push, pull_request, schedule).

Ans: push/pull_request trigger CI on code changes; schedule (cron) can trigger periodic jobs like nightly model retraining or scheduled evaluation runs.

Q8. How would you version your Docker images in a CI/CD pipeline for traceability?

Ans: Tag images with the git commit SHA (and optionally semantic version) rather than just 'latest', so every deployed image is traceable back to an exact code state.

Q9. What is Infrastructure as Code and why does it matter for reproducible ML environments?

Ans: IaC (Terraform, CloudFormation) defines infrastructure in versioned config files, ensuring environments can be recreated identically instead of manually configured and drifting over time.

Q10. Explain the concept of a rollback strategy in CI/CD for a production RAG chatbot.

Ans: Keep previous container image tags available and have a scripted or one-click deployment revert to the last known-good version if the new release causes errors or quality regressions.

Q11. How do secrets management tools (Vault, AWS Secrets Manager) integrate into a CI/CD pipeline?

Ans: The pipeline fetches secrets (API keys, DB credentials) at deploy time from the secrets manager instead of storing them in code or CI config files, reducing leak risk.

Q12. What's the difference between a CI pipeline test stage and a smoke test post-deployment?

Ans: Test stage validates code correctness before deployment (unit/integration tests); smoke tests run after deployment against the live environment to confirm the service actually starts and responds correctly.

Q13. Explain how you'd set up automated linting/formatting (black, flake8, ruff) as a CI gate.

Ans: Add a CI job that runs the linter/formatter in check mode on every PR; fail the build if violations are found, enforcing consistent code style before merge.

Q14. How would you handle database/vector index migrations safely within a CI/CD pipeline?

Ans: Run migrations as a separate, idempotent pipeline step before the new app version goes live, with backward-compatible schema changes so the old version can still run during rollout.

Q15. What is a matrix build in CI/CD and when would you use it for an AI project?

Ans: A matrix build runs the same job across multiple configurations (e.g. Python versions, OS, GPU vs CPU) in parallel, ensuring compatibility across environments.

Q16. Explain artifact caching in CI/CD pipelines for faster ML builds.

Ans: Cache dependencies (pip/npm packages, Docker layers) between pipeline runs so unchanged dependencies aren't re-downloaded/re-installed every single run.

Q17. How would you implement approval gates before deploying a new LLM prompt version to production?

Ans: Require a manual approval step (e.g. GitHub Environments protection rules) after automated tests pass, so a human reviews eval results before promoting to production.

Q18. What's the difference between GitOps and traditional CI/CD?

Ans: GitOps treats a Git repository as the single source of truth for desired infrastructure/app state, with an operator (e.g. ArgoCD) continuously reconciling the cluster to match it, rather than pipelines pushing changes directly.

Q19. How do you monitor deployment health automatically and trigger auto-rollback?

Ans: Hook deployment to metrics/alerts (error rate, latency, health check failures) and configure the CD tool to automatically revert to the previous version if thresholds are breached post-deploy.

Q20. Explain how you would structure CI/CD for a monorepo containing multiple agent microservices.

Ans: Use path-based triggers so only changed services rebuild/deploy, combined with shared reusable workflow templates to avoid duplicating pipeline logic across services.

Q1. What is a Pod in Kubernetes and why not run containers directly?

Ans: A Pod is the smallest deployable unit, wrapping one or more tightly-coupled containers that share network/storage - Kubernetes manages pods (not raw containers) for scheduling, scaling and networking.

Q2. Explain the difference between a Deployment and a StatefulSet - which fits a vector DB?

Ans: Deployment manages stateless, interchangeable replicas; StatefulSet gives stable network identities and persistent storage per pod, which fits stateful services like vector databases.

Q3. How does a Kubernetes Service enable communication between an LLM API pod and a frontend pod?

Ans: A Service provides a stable virtual IP/DNS name that load-balances traffic across the matching pods, so callers don't need to track individual pod IPs that change on restarts.

Q4. What is a Horizontal Pod Autoscaler and how would you configure it for an LLM inference service?

Ans: HPA automatically scales pod replica count based on metrics like CPU/memory or custom metrics (e.g. request queue length), adding replicas under high inference load and scaling down when idle.

Q5. Explain the difference between a ConfigMap and a Secret.

Ans: ConfigMap stores non-sensitive configuration data as key-value pairs; Secret stores sensitive data (API keys, passwords) and is base64-encoded with tighter access controls.

Q6. How would you expose a GPU to a Kubernetes pod running a local embedding model?

Ans: Install the NVIDIA device plugin on the cluster, then request `nvidia.com/gpu` in the pod's resource limits so the scheduler places it on a GPU-enabled node.

Q7. What is a readiness probe vs a liveness probe, and how would you use them for a FastAPI agent service?

Ans: Readiness probe determines if a pod is ready to receive traffic (removed from Service endpoints if failing); liveness probe determines if a pod should be restarted (deadlocked/hung) - both typically hit a `/health` endpoint.

Q8. Explain Kubernetes namespaces and how you'd use them to separate dev/staging/prod agent deployments.

Ans: Namespaces logically partition cluster resources; you'd deploy identical manifests into dev/staging/prod namespaces with different configs/secrets, isolating them within the same cluster.

Q9. What is an Ingress and how does it differ from a NodePort Service?

Ans: Ingress provides HTTP(S) routing (host/path based) and TLS termination through a single entry point; NodePort directly exposes a service on a static port on every node, which is less flexible and less secure.

Q10. How would you roll out a new version of an LLM microservice with zero downtime?

Ans: Use a Deployment's RollingUpdate strategy, which gradually replaces old pods with new ones while keeping enough replicas available, combined with readiness probes to avoid routing traffic to unready pods.

Q11. Explain resource requests vs limits in a Kubernetes pod spec for a memory-heavy embedding job.

Ans: Requests are the guaranteed minimum resources reserved for scheduling; limits are the hard cap - exceeding memory limits gets a pod OOMKilled, so limits must be set carefully for memory-intensive workloads.

Q12. What is a Kubernetes Job vs CronJob, and how does this relate to scheduled model retraining?

Ans: A Job runs a task to completion once; a CronJob runs Jobs on a schedule (cron syntax) - fits recurring tasks like nightly embedding re-indexing or scheduled fine-tuning triggers.

Q13. Explain how you'd implement blue-green deployment natively in Kubernetes.

Ans: Deploy the new version as a separate Deployment with its own label, then switch the Service's selector to point traffic to the new version's pods once verified - and switch back instantly if issues arise.

Q14. What is a sidecar container and how might it help with logging/monitoring an agent pod?

Ans: A sidecar runs alongside the main container in the same pod, sharing its network/storage - e.g. a log-shipping sidecar can tail and forward the main container's logs without modifying the app itself.

Q15. How does Kubernetes handle secret rotation for something like a Groq/OpenAI API key?

Ans: Secrets can be updated in the cluster (kubectl apply / secret store CSI driver), but pods need to be restarted or use a mechanism that watches for mounted secret changes to pick up the new value.

Q16. Explain the role of etcd in a Kubernetes cluster.

Ans: etcd is the distributed key-value store holding the cluster's entire state and configuration - the control plane (API server) reads/writes to it as the single source of truth.

Q17. What is a PersistentVolumeClaim and how would it apply to storing Chroma/FAISS index files?

Ans: A PVC requests storage from the cluster (backed by a PersistentVolume); mounting it into a pod ensures index files persist across pod restarts/rescheduling instead of being lost.

Q18. How would you debug a pod stuck in CrashLoopBackOff for an LLM service?

Ans: Run kubectl logs and kubectl describe pod to check events/exit codes, verify environment variables/secrets are correctly injected, and check resource limits aren't causing OOMKills.

Q19. Explain the difference between kubectl apply and kubectl create.

Ans: kubectl create only creates new resources and errors if they exist; kubectl apply declaratively creates or updates resources to match the manifest, making it idempotent and safer for CI/CD.

Q20. What is a taint and toleration, and how would you dedicate GPU nodes to inference pods only?

Ans: Taints repel pods from a node unless the pod has a matching toleration - tainting GPU nodes ensures only pods explicitly tolerating (and requesting) GPU resources get scheduled there.

06 MLflow & DVC (Experiment Tracking / Versioning)

20 Questions

Q1. What problem does MLflow Tracking solve in an LLM/RAG project?

Ans: It logs parameters, metrics, artifacts and code versions for every experiment run, so you can compare prompt variants, chunking strategies, or model configs systematically instead of relying on memory/spreadsheets.

Q2. Explain the four main components of MLflow.

Ans: Tracking (logging experiments), Projects (packaging reproducible runs), Models (standard packaging/deployment format), and Model Registry (versioning and lifecycle stages of models).

Q3. How would you use MLflow to compare different RAG retrieval strategies (top-k values, chunk sizes)?

Ans: Log each configuration as a separate run with its parameters (chunk_size, top_k, embedding_model) and evaluation metrics (faithfulness, recall) so you can filter/sort runs in the MLflow UI to find the best config.

Q4. What is the MLflow Model Registry and why does it matter for promoting an LLM pipeline to production?

Ans: It tracks model versions through stages (Staging, Production, Archived) with lineage and approval history, giving controlled, auditable promotion instead of manually swapping files.

Q5. Explain the difference between DVC and Git.

Ans: Git tracks code changes efficiently via diffs; DVC tracks large data/model files by storing lightweight pointer files in Git while the actual data lives in remote storage (S3, GDrive, etc.), making large-file versioning practical.

Q6. How does DVC enable reproducible ML pipelines?

Ans: DVC pipelines (dvc.yaml) define stages with dependencies and outputs; running dvc repro re-executes only the stages whose inputs changed, guaranteeing consistent, cacheable results.

Q7. What is a DVC remote and why would you configure one for a team project?

Ans: A DVC remote is external storage (S3, GDrive, Azure Blob) where actual data/model files are pushed/pulled, letting teammates share large files without bloating the Git repo.

Q8. Explain how you'd log a fine-tuned LLM's hyperparameters and evaluation scores using MLflow in a Python script.

Ans: Wrap the training/eval code with mlflow.start_run(), then call mlflow.log_param() for hyperparameters and mlflow.log_metric() for scores, with mlflow.log_artifact() for saved model files or plots.

Q9. What's the difference between DVC pipelines and MLflow Projects?

Ans: DVC pipelines focus on data/dependency-driven reproducibility (like a Makefile for data science); MLflow Projects focus on packaging code with its environment for reproducible execution across different platforms.

Q10. How would you track dataset versions used for building your vector store embeddings?

Ans: Use `dvc add` on the raw dataset/embeddings folder, commit the resulting `.dvc` pointer file to Git, and push the actual data to a DVC remote - any commit checkout restores the exact matching data version.

Q11. Explain MLflow's autologging feature and its limitations for LLM-based workflows.

Ans: Autologging automatically captures parameters/metrics for supported frameworks (scikit-learn, PyTorch) with minimal code, but for custom LLM/RAG pipelines you typically need manual logging since there's no standard autolog integration.

Q12. How would you set up a CI check that fails if a new model version underperforms the current production model in the registry?

Ans: Fetch the current Production-stage model's recorded metrics from the Registry, run the same eval suite against the new candidate, and fail the pipeline if the candidate's metric is worse than a defined threshold.

Q13. What is data drift and how would MLflow/monitoring tools help detect it in an LLM app?

Ans: Data drift is when incoming data's distribution changes over time from what the system was built for; logging input feature statistics per run over time (or using dedicated monitoring tools) helps detect this shift early.

Q14. Explain how DVC handles pipeline caching to avoid redundant recomputation.

Ans: DVC hashes each stage's dependencies and parameters; if nothing changed since the last run, it restores cached outputs instead of re-executing that stage, saving compute time.

Q15. How would you structure an MLflow experiment hierarchy for a multi-agent project with several sub-agents?

Ans: Create one parent experiment per project, and either nested runs or clearly named runs per agent (e.g. 'retriever-agent-v2', 'summarizer-agent-v1') so metrics stay organized and comparable per component.

Q16. What is model lineage and why does it matter for compliance/debugging in production LLM systems?

Ans: Model lineage traces exactly which data, code version, and parameters produced a given model artifact - critical for debugging regressions and for compliance audits requiring reproducibility.

Q17. Explain the difference between logging metrics as a single value vs a step-wise time series in MLflow.

Ans: A single value records one final number (e.g. final eval score); step-wise logging records a metric at each step/epoch, letting you visualize training curves over time in the MLflow UI.

Q18. How would DVC help you roll back to a previous version of your embeddings after a bad re-indexing run?

Ans: Since each dataset/index version is tied to a Git commit via `.dvc` files, checking out the previous commit and running `dvc checkout` restores exactly that prior version of the embeddings.

Q19. What are DVC experiments (dvc exp) and how do they differ from full pipeline commits?

Ans: dvc exp run lets you try lightweight, disposable pipeline variations without committing to Git each time - only promoting an experiment to a real commit once you're satisfied with results.

Q20. How would you integrate MLflow tracking directly into a LangGraph agent pipeline for observability?

Ans: Wrap each node/step execution with `mlflow.start_run()` (or nested runs), logging inputs, outputs, latencies and any evaluation scores per node so you can trace exactly how the agent's decision path performed.

Q1. What is the difference between a base LLM and an instruction-tuned LLM?

Ans: A base model is trained purely to predict next tokens on raw text; an instruction-tuned model is further fine-tuned (often with RLHF) on instruction-response pairs to follow commands and align with user intent.

Q2. Explain temperature and top-p sampling - how do they differ?

Ans: Temperature scales the probability distribution's sharpness (higher = more random); top-p (nucleus sampling) restricts sampling to the smallest set of tokens whose cumulative probability exceeds p - they can be combined.

Q3. What is the context window and why does it matter for RAG design?

Ans: It's the maximum number of tokens (input + output) the model can process at once - RAG must fit retrieved chunks plus the question plus generated answer within this limit, forcing careful chunk selection.

Q4. Explain the difference between embeddings and LLM outputs.

Ans: Embeddings are dense numeric vectors representing semantic meaning for similarity comparison; LLM outputs are generated tokens/text representing a response - embeddings power retrieval, generation powers the answer.

Q5. What is hallucination in LLMs and what techniques reduce it?

Ans: Hallucination is generating confident but factually incorrect content; RAG grounding, lower temperature, citation requirements, and fact-checking/self-verification steps all help reduce it.

Q6. Explain the difference between fine-tuning and prompt engineering - when would you choose each?

Ans: Fine-tuning updates model weights on custom data (better for consistent domain-specific behavior at scale); prompt engineering shapes behavior via input design only (faster, cheaper, no training infra needed) - start with prompting, fine-tune only if it plateaus.

Q7. What is LoRA and why is it preferred over full fine-tuning for most teams?

Ans: LoRA (Low-Rank Adaptation) freezes the base model and trains small low-rank matrices injected into layers, drastically cutting the number of trainable parameters and GPU memory needed versus full fine-tuning.

Q8. Explain the transformer's self-attention mechanism at a high level.

Ans: Each token computes Query, Key and Value vectors; attention scores (from Query-Key similarity) determine how much each other token's Value contributes to the current token's new representation.

Q9. What is the difference between encoder-only, decoder-only, and encoder-decoder architectures?

Ans: Encoder-only (like BERT) is best for understanding/classification tasks; decoder-only (like GPT) is best for autoregressive generation; encoder-decoder (like T5) is suited for sequence-to-sequence tasks like translation/summarization.

Q10. Explain what a system prompt is and why it's not a security boundary.

Ans: A system prompt sets initial instructions/persona for the model, but since everything is just text in the context, a sufficiently crafted user input (prompt injection) can sometimes override or leak it - it's guidance, not a hard security wall.

Q11. What is prompt injection and how would you mitigate it in an agent that uses external tools?

Ans: It's malicious input designed to hijack the model's instructions - mitigations include input sanitization, strict system/user role separation, output validation, and limiting what tools/actions the model can autonomously trigger.

Q12. Explain the difference between zero-shot, few-shot, and chain-of-thought prompting.

Ans: Zero-shot gives no examples, relying purely on instructions; few-shot provides a handful of example input-output pairs to guide format/style; chain-of-thought explicitly asks the model to reason step-by-step before answering.

Q13. What is model quantization (e.g. GGUF, 4-bit) and its tradeoff?

Ans: Quantization reduces the precision of model weights (e.g. FP16 to INT4), shrinking memory/compute needs and enabling local deployment, at the cost of some accuracy/quality degradation.

Q14. Explain token vs word - why does an LLM 'think' in tokens?

Ans: A token is a sub-word unit from the model's tokenizer (not always a whole word); models operate on token sequences because it balances vocabulary size with the ability to represent rare/unseen words via sub-word pieces.

Q15. What is the difference between greedy decoding and beam search?

Ans: Greedy decoding picks the single highest-probability token at each step (fast, can be suboptimal); beam search keeps multiple candidate sequences in parallel and picks the overall best one at the end (better quality, more compute).

Q16. Explain what 'grounding' means in the context of LLM applications.

Ans: Grounding means constraining the model's output to be based on specific provided evidence/context (like retrieved documents) rather than purely its parametric memory, reducing hallucination.

Q17. What is the difference between a reward model and the policy model in RLHF?

Ans: The reward model is trained to score how good/aligned a response is (learned from human preference data); the policy model (the LLM itself) is then optimized via reinforcement learning to maximize that reward model's score.

Q18. Explain why longer context windows don't automatically solve the 'lost in the middle' problem.

Ans: Research shows models often attend better to information at the very start/end of the context than in the middle, so simply stuffing more text into a huge context window doesn't guarantee all of it is used effectively.

Q19. What is speculative decoding and how does it speed up LLM inference?

Ans: A small, fast draft model proposes several tokens ahead, and the larger target model verifies them in parallel in a single forward pass, accepting correct guesses - this reduces the number of expensive sequential passes needed.

Q20. Explain function calling / tool calling in LLMs and how the model decides to invoke a tool.

Ans: The model is given a schema of available tools/functions; based on training and the prompt, it outputs a structured call (name + arguments) when it determines a tool would help answer, which your code then executes and feeds the result back.

Q21. What is the difference between an LLM agent and a simple LLM chain?

Ans: A chain executes a fixed, predetermined sequence of steps; an agent dynamically decides which tools/steps to take next based on intermediate results, in a loop, until it determines the task is complete.

Q22. Explain catastrophic forgetting in the context of fine-tuning an LLM.

Ans: Fine-tuning heavily on a narrow new dataset can cause the model to lose previously learned general capabilities - mitigated using techniques like LoRA, lower learning rates, or mixing in original training data.

Q23. What is the KV cache in transformer inference and why does it matter for latency?

Ans: The KV (Key-Value) cache stores previously computed attention keys/values for earlier tokens so they aren't recomputed at every new generation step, significantly speeding up autoregressive decoding.

Q24. Explain the difference between open-weight and closed/proprietary LLMs (e.g. Llama vs GPT).

Ans: Open-weight models release their trained weights for self-hosting/fine-tuning (e.g. Llama, Mistral); closed models are only accessible via API with no weight access (e.g. GPT-4, Claude) - affecting cost, control, and data privacy tradeoffs.

Q25. What is model distillation and how might you use it to create a faster production LLM?

Ans: Distillation trains a smaller 'student' model to mimic a larger 'teacher' model's outputs, retaining much of its capability at a fraction of the size/latency/cost - useful for cutting production inference costs.

Q1. What problem does RAG solve that fine-tuning alone doesn't?

Ans: RAG lets the model access fresh, external, or proprietary knowledge at query time without retraining, and provides traceable sources - fine-tuning bakes knowledge into weights and quickly goes stale.

Q2. Explain the full RAG pipeline end to end.

Ans: Documents are chunked, embedded, and stored in a vector DB; at query time, the query is embedded, top-k similar chunks are retrieved, and those chunks plus the question are passed to the LLM to generate a grounded answer.

Q3. What is chunking and why does chunk size/overlap matter?

Ans: Chunking splits documents into smaller retrievable pieces; too-large chunks dilute relevance and waste context, too-small chunks lose surrounding context - overlap prevents losing information split across chunk boundaries.

Q4. Explain the difference between semantic search and keyword (BM25) search in RAG.

Ans: Semantic search matches based on embedding similarity (captures meaning/synonyms); BM25 matches based on exact/weighted term frequency (captures precise keywords/names) - hybrid search combines both for better recall.

Q5. What is hybrid search and when would you use it over pure vector search?

Ans: Hybrid search combines vector similarity with keyword-based scoring (like BM25), which helps when queries contain exact terms (IDs, names, codes) that pure semantic embeddings might not weight strongly enough.

Q6. Explain re-ranking in a RAG pipeline and why it improves answer quality.

Ans: After initial retrieval fetches a broader candidate set, a re-ranker (often a cross-encoder) scores each candidate more precisely against the query, reordering results so the most truly relevant chunks are prioritized before generation.

Q7. What's the difference between a bi-encoder and a cross-encoder for retrieval?

Ans: A bi-encoder embeds query and document independently for fast similarity search at scale; a cross-encoder jointly processes query+document together for much higher accuracy but is too slow to run on the full corpus, so it's used for re-ranking a smaller candidate set.

Q8. How would you evaluate a RAG system's quality (faithfulness, relevance)?

Ans: Use metrics like faithfulness (does the answer match retrieved context), answer relevance, and context precision/recall - frameworks like RAGAS or an LLM-as-judge can automate scoring against a labeled test set.

Q9. Explain the 'lost in the middle' problem specifically in RAG and how to counter it.

Ans: Relevant chunks placed in the middle of a long context are more likely to be under-utilized by the model; counter this by keeping context concise, placing the most relevant chunks near the start/end, or using re-ranking to push top chunks to the front.

Q10. What is metadata filtering in vector search and why is it useful?

Ans: Metadata filtering lets you narrow the vector search to a subset of documents (by date, source, user, category) using structured filters alongside the similarity search, improving precision for multi-tenant or time-sensitive data.

Q11. Explain the difference between naive RAG and agentic RAG.

Ans: Naive RAG does a single retrieve-then-generate pass; agentic RAG allows the model to iteratively decide to re-query, use different tools, or refine its search strategy based on intermediate results before producing a final answer.

Q12. What is query expansion/rewriting and why would you use it before retrieval?

Ans: It reformulates or expands the user's original query (synonyms, sub-questions, clarified intent) before embedding it, improving the chance of retrieving relevant chunks that don't share exact wording with the original query.

Q13. Explain contextual/parent-child chunking strategy in RAG.

Ans: Small chunks are used for precise embedding-based matching, but when retrieved, their larger parent chunk/section is returned to the LLM - balancing retrieval precision with sufficient surrounding context for generation.

Q14. What is the difference between dense retrieval and sparse retrieval?

Ans: Dense retrieval uses learned embedding vectors capturing semantic meaning; sparse retrieval uses high-dimensional term-frequency-based representations (like TF-IDF/BM25) capturing exact lexical matches.

Q15. How would you handle multi-hop questions that require combining information from multiple chunks?

Ans: Use iterative/agentic retrieval where the model can issue follow-up queries based on what it's learned so far, or decompose the question into sub-questions, retrieving and answering each before combining into a final answer.

Q16. Explain how you would prevent a RAG system from answering when no relevant context is retrieved.

Ans: Set a similarity score threshold below which no chunks qualify as relevant, and explicitly instruct/constrain the LLM to respond with 'I don't know' or similar rather than fabricating an answer from its own knowledge.

Q17. What is the role of an embedding model's dimensionality and how does it affect a vector DB's performance?

Ans: Higher dimensions can capture more nuance but increase storage size and search latency; choosing dimensionality is a tradeoff between retrieval quality and infrastructure cost/speed.

Q18. Explain how you'd design a RAG system to cite its sources accurately.

Ans: Track chunk-to-source metadata through the pipeline, instruct the LLM to reference specific chunk IDs/sources in its answer, and post-process to map those references back to actual document citations shown to the user.

Q19. What's the difference between HNSW and IVF indexing algorithms in vector databases?

Ans: HNSW builds a navigable small-world graph for fast, high-recall approximate search with higher memory use; IVF (Inverted File Index) clusters vectors into partitions searched selectively, often using less memory but with different recall/speed tradeoffs.

Q20. How would you reduce latency in a production RAG pipeline serving thousands of users?

Ans: Cache frequent queries/embeddings, use an efficient ANN index (HNSW), batch embedding calls, use a smaller/faster re-ranker, and consider async parallel retrieval + generation stages.

Q21. Explain corrective RAG (CRAG) and self-RAG concepts.

Ans: Corrective RAG evaluates retrieved documents' relevance and triggers a fallback (like a web search) if retrieval quality is poor; Self-RAG trains the model to critique and decide when to retrieve, and to self-assess its own generated answer's quality.

Q22. What is the cold-start problem in RAG for a brand-new knowledge base with sparse documents?

Ans: With too few documents, embeddings may not capture enough semantic variety for accurate retrieval; mitigations include using a strong general-purpose embedding model, adding synthetic Q&A pairs, or falling back to keyword search until more data exists.

Q23. Explain how you'd version control a RAG knowledge base as documents get updated frequently.

Ans: Track document versions/timestamps as metadata, re-embed and upsert only changed documents, and maintain a way to invalidate/remove stale chunks so retrieval never surfaces outdated information.

Q24. What is the difference between RAG and long-context LLMs - will huge context windows replace RAG?

Ans: Long-context models can fit more raw text directly, but RAG remains more cost-efficient, scalable to massive/dynamic knowledge bases, and better for precise retrieval + citation - most production systems combine both rather than replacing one with the other.

Q25. Explain how you would design an evaluation dataset (golden set) for testing your RAG pipeline before each deployment.

Ans: Curate a diverse set of representative questions with verified correct answers and expected source chunks, covering edge cases (no-answer scenarios, multi-hop questions), then run this set through CI on every pipeline change to catch regressions.

Q1. How would you design a scalable RAG-based customer support chatbot for 100k+ daily users?

Ans: Load-balanced FastAPI backend with async request handling, a horizontally-scaled vector DB, Redis caching for frequent queries, rate-limited LLM API calls with retries/fallback models, and async logging/monitoring - all containerized and auto-scaled via Kubernetes HPA.

Q2. Explain how you'd design a multi-agent system for automating a sales pipeline.

Ans: A supervisor agent routes tasks to specialist agents (lead qualification, email drafting, CRM update); each agent has scoped tools and memory, communicating through a shared state/message bus, with the supervisor aggregating results and handling failures.

Q3. How would you architect an LLM system to handle both real-time chat and long-running batch document processing?

Ans: Separate services: a low-latency synchronous API for chat, and an async job queue (Celery/SQS) for batch processing that workers pull from independently - both sharing the same underlying model/vector DB infrastructure but scaled separately based on their different load patterns.

Q4. Design a caching strategy for an LLM API to reduce cost without serving stale answers.

Ans: Cache by a hash of (normalized prompt + relevant parameters); use semantic caching (embedding similarity) for near-duplicate queries, with a TTL and invalidation trigger when underlying source documents change.

Q5. How would you design rate limiting and cost control for a multi-tenant LLM SaaS product?

Ans: Per-tenant token/request quotas enforced at the API gateway, tiered plans with different rate limits, real-time usage tracking, and automatic throttling/queueing (or upgrade prompts) when a tenant approaches their quota.

Q6. Explain how you'd design a fallback strategy when your primary LLM provider (e.g. Groq) has an outage.

Ans: Implement a provider abstraction layer with automatic failover to a secondary provider (e.g. OpenAI/Gemini) on error/timeout, health-check the primary periodically, and log/alert on failovers for visibility.

Q7. How would you design an observability stack for a production agentic AI system?

Ans: Structured logging of every agent step/tool call, distributed tracing (e.g. OpenTelemetry) to follow a request across services, dashboards for latency/cost/error-rate metrics, and an eval pipeline sampling live traffic for quality regression detection.

Q8. Design a system to safely let an LLM agent execute code or shell commands.

Ans: Run execution in an isolated, resource-limited sandbox/container with no network access (or an allowlist), strict timeouts, no access to secrets/host filesystem, and human-in-the-loop approval for higher-risk actions.

Q9. How would you design a memory system for a long-running conversational agent that exceeds the context window?

Ans: Use short-term memory (recent turns in context) combined with long-term memory (a vector store of summarized past interactions), retrieving relevant past context on demand rather than keeping the entire history in every prompt.

Q10. Explain how you'd design an A/B testing framework for comparing two different prompt versions in production.

Ans: Route a percentage of live traffic to each prompt variant, log outcomes (user feedback, task completion, eval scores) tagged by variant, and use statistical significance testing before fully rolling out the winning version.

Q11. How would you design a system to detect and prevent prompt injection attacks at scale?

Ans: Combine input sanitization/pattern detection, strict separation of system vs untrusted user/tool content, an LLM-based injection classifier as a pre-filter, and constrained output/tool permissions so even a successful injection has limited blast radius.

Q12. Design a document ingestion pipeline that handles PDFs, images, and scanned documents for a RAG system.

Ans: A pipeline with format detection, OCR fallback for scanned/image content, layout-aware parsing (tables/headers), chunking, embedding, and metadata tagging - orchestrated as an async worker queue so ingestion scales independently of the query-serving path.

Q13. How would you architect cost monitoring/alerts for LLM token usage across multiple teams in an organization?

Ans: Tag every LLM call with team/project metadata, stream usage events to a metrics store, build per-team dashboards, and set budget-threshold alerts (or hard cutoffs) that notify or throttle before costs spiral.

Q14. Explain how you'd design a human-in-the-loop review system for high-stakes agent actions (e.g. sending emails).

Ans: Agent actions above a risk threshold are queued for human approval via a review UI before execution, with approved/rejected decisions logged and optionally fed back to improve the agent's future decision-making.

Q15. How would you design disaster recovery for a vector database holding your entire knowledge base?

Ans: Regular automated backups/snapshots to durable storage, a documented restore procedure tested periodically, and ideally a standby replica in a different region/zone for fast failover during an outage.

Q16. Design a system for continuously evaluating a production LLM app's quality without manual review of every response.

Ans: Automatically sample a subset of live interactions, score them using an LLM-as-judge against defined rubrics (accuracy, tone, safety), track scores over time on a dashboard, and alert when quality metrics degrade beyond a threshold.

Q17. How would you design a multi-region deployment for an LLM app to minimize latency for global users?

Ans: Deploy regional API gateways/edge caches close to users, route requests to the nearest healthy region, replicate the vector DB read-layer regionally while keeping a consistent write path, and use a global LLM provider endpoint if regional model hosting isn't available.

Q18. Explain how you'd design a system to handle streaming LLM responses to the frontend in real time.

Ans: Use Server-Sent Events or WebSockets so the backend streams generated tokens to the client as they arrive from the LLM API, updating the UI incrementally instead of waiting for the full response.

Q19. How would you design a feedback loop where user thumbs-up/down ratings improve future RAG retrieval quality?

Ans: Log ratings tied to the query, retrieved chunks, and generated answer; periodically analyze low-rated cases to identify retrieval gaps, and use this signal to tune chunking, re-ranking weights, or add missing documents to the knowledge base.

Q20. Design an architecture that lets non-technical users create their own custom AI agents (a no-code agent builder).

Ans: A visual workflow builder (drag-drop nodes) mapping to an underlying agent graph (like LangGraph), a library of pre-built tool connectors, a sandboxed execution engine per user-created agent, and versioning/rollback for published agent configurations.

Q21. How would you design capacity planning for GPU inference servers handling unpredictable traffic spikes?

Ans: Combine a baseline of reserved capacity for steady-state load with auto-scaling burst capacity (spot/on-demand instances) triggered by queue depth or latency metrics, plus request queueing/backpressure to smooth out extreme spikes without over-provisioning.

Q22. Explain trade-offs between building a monolithic agent vs a microservices-based multi-agent architecture.

Ans: Monolithic is simpler to build/debug initially but harder to scale/maintain as complexity grows; microservices-based multi-agent allows independent scaling, technology choices, and fault isolation per agent, at the cost of added orchestration and network complexity.

Q23. How would you design a system to ensure PII doesn't leak into LLM prompts or logs?

Ans: Run PII detection/redaction on inputs before they reach the LLM or get logged, use scoped data access per request, and audit logs regularly with automated scanning tools to catch any leakage.

Q24. Design a system for testing an agent's behavior against adversarial or edge-case user inputs before release.

Ans: Maintain a growing red-team test suite covering prompt injection attempts, ambiguous/malicious requests, and edge cases, running it automatically in CI against every prompt/model change and requiring passing scores before deployment.

Q25. How would you design your system so switching LLM providers (e.g. Groq to a self-hosted model) requires minimal code changes?

Ans: Abstract all LLM calls behind a common interface/adaptor layer (matching something like LangChain's model abstraction), keep prompts and business logic provider-agnostic, and centralize provider-specific configuration so swapping providers is a config change, not a rewrite.